

PROGRAMACIÓN AVANZADA

Patrones de Diseño



Ing. Matias Macrino

PATRONES DE DISEÑO

- ▶ ¿Qué es un patrón de diseño?:
 - ▶ “Son el esqueleto de las soluciones a problemas comunes en el desarrollo de Software”
 - ▶ Dicho en pocas palabras, un patrón de diseño es una solución a un problema de diseño.
 - ▶ Para considerarse una solución como patrón se debe de comprobar su **efectividad** y que el mismo sea **reutilizable**.

PATRONES DE DISEÑO

- ▶ Tipos de patrones de diseño:
- ▶ **Patrones creacionales:** son aquellos que se encargan de la inicialización y configuración de objetos.
- ▶ **Patrones estructurales:** son aquellos que separan la interfaz de la implementación. Se ocupan de como las clases y objetos de agrupan, para formar estructuras mas grandes.
- ▶ **Patrones de comportamiento:** son aquellos que se ocupan mas que describir objetos o clases, describen la comunicación entre ellos.

PATRONES DE DISEÑO

- ▶ Patrones creacionales:

- ▶ DAO (Data Acces Object):

- ▶ La idea de este patrón es sencilla, resolver netamente el acceso a los datos, que básicamente tiene que ver con la gestión de diversas fuentes de datos.
- ▶ Con este nos abstraemos de la forma de acceder a los datos.

PATRONES DE DISEÑO

- ▶ Patrones creacionales:

- ▶ DAO ¿Cómo usarlo? :

- ▶ Primero es crear una entidad que nos represente los datos de nuestro negocio (**POJO, DTO**)
- ▶ Segundo generar la interfaz **DAO**, la cual solamente contendrá los métodos necesario para obtener y almacenar los datos (**CRUD**), no tiene que contener ningún tipo de relación con el tipo de almacenamiento que se vaya a utilizar.
- ▶ Tercero una vez que tengamos definidas las operaciones, es momento de construir su implementación **DAOImplement**, la cual si contendrá una relación directa con el tipo de almacenamiento que utilizara nuestra aplicación.

PATRONES DE DISEÑO

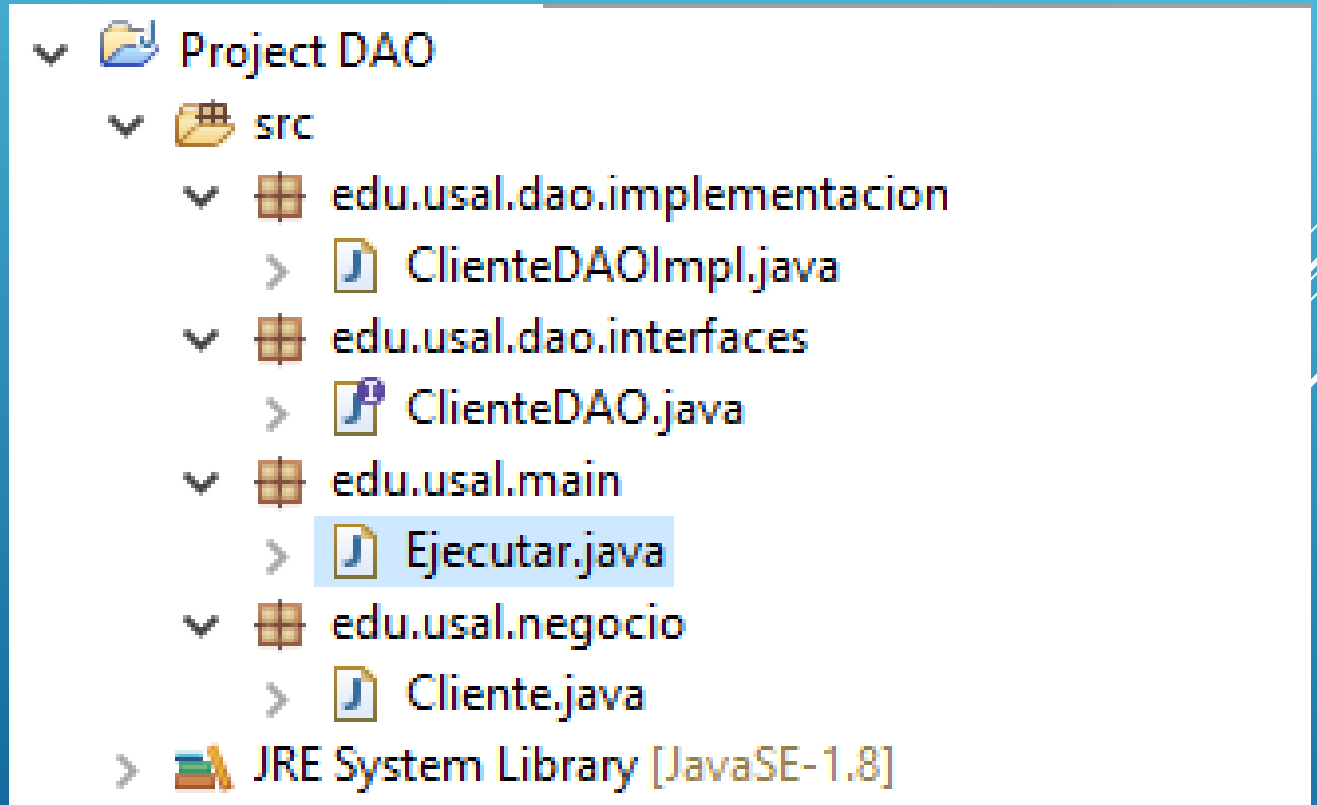
- ▶ Patrones creacionales:

- ▶ DAO ¿Cómo usarlo? Ejemplo :

- ▶ Nuestra entidad de negocio, será creada en una clase del tipo **POJO**, lo único que contendrá nuestra clase son los correspondientes setters and getters de los atributos y sus constructores.
- ▶ Nuestra interfaz contendrá las operaciones necesarias, por ejemplo obtener todas las personas y buscar personas por ID.
- ▶ Nuestra aplicación contendrá acceso a dos motores distintos de datos, por lo cual tendremos dos implementación correspondientes a una sola interfaz, la que fue creada en el segundo paso.

PATRONES DE DISEÑO

- ▶ Patrones creacionales:
- ▶ DAO Estructura de Proyecto:



PATRONES DE DISEÑO

- ▶ Patrones creacionales:
- ▶ DAO Interface:

```
public interface ClienteDAO {  
    public List<Cliente> obtenerClientes();  
    public Cliente obtenerCliente(int id);  
    public void actualizarCliente(Cliente cliente);  
    public void eliminarCliente(Cliente cliente);  
}
```

PATRONES DE DISEÑO

► Patrones creacionales:

► DAO Implementation:

```
public class ClienteDAOImpl implements ClienteDAO{

    List<Cliente> clientes;

    public ClienteDAOImpl() {
        clientes = new ArrayList<>();
        clientes.add(new Cliente(0, "Javier", "Molina"));
        clientes.add(new Cliente(1, "Lillian", "Álvarez"));
    }

    @Override
    public List<Cliente> obtenerClientes() {
        return clientes;
    }

    @Override
    public Cliente obtenerCliente(int id) {
        return clientes.get(id);
    }

    @Override
    public void actualizarCliente(Cliente cliente) {
        clientes.get(cliente.getId()).setNombre(cliente.getNombre());
        clientes.get(cliente.getId()).setApellido(cliente.getApellido());
        System.out.println("Cliente con id: "+cliente.getId()+" actualizado satisfactoriamente");
    }

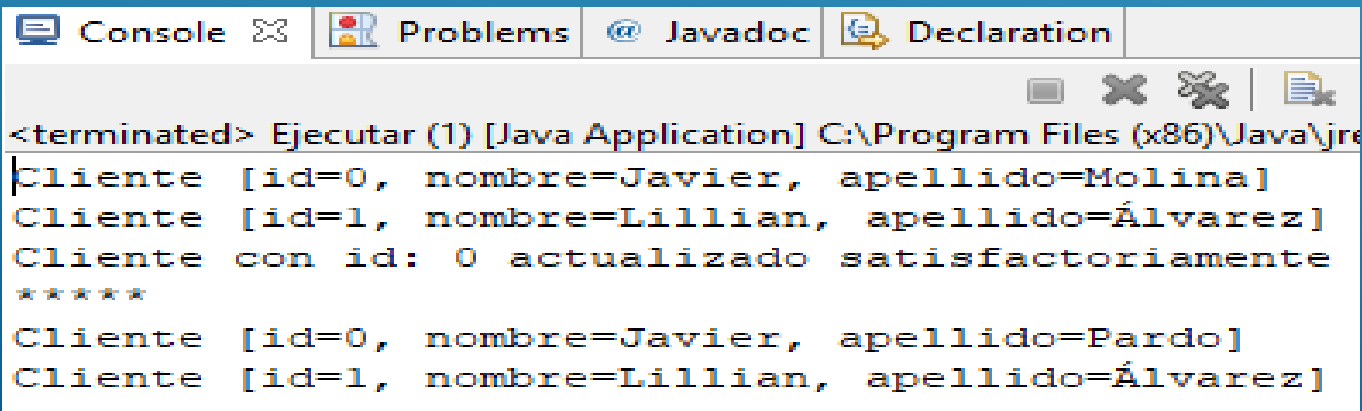
    @Override
    public void eliminarCliente(Cliente cliente) {
        clientes.remove(cliente.getId());
        System.out.println("Cliente con id: "+cliente.getId()+" eliminado satisfactoriamente");
    }
}
```

PATRONES DE DISEÑO

► Patrones creacionales:

► DAO Ejecución:

```
public class Ejecutar {  
  
    public static void main(String[] args) {  
        ClienteDAO clienteDao = new ClienteDAOImpl();  
  
        clienteDao.obtenerClientes().forEach(System.out::println);  
  
        Cliente cliente = clienteDao.obtenerCliente(0);  
        cliente.setApellido("Pardo");  
  
        clienteDao.actualizarCliente(cliente);  
  
        System.out.println("*****");  
        clienteDao.obtenerClientes().forEach(System.out::println);  
    }  
}
```



```
<terminated> Ejecutar (1) [Java Application] C:\Program Files (x86)\Java\jre  
Cliente [id=0, nombre=Javier, apellido=Molina]  
Cliente [id=1, nombre=Lillian, apellido=Álvarez]  
Cliente con id: 0 actualizado satisfactoriamente  
*****  
Cliente [id=0, nombre=Javier, apellido=Pardo]  
Cliente [id=1, nombre=Lillian, apellido=Álvarez]
```

PATRONES DE DISEÑO

- ▶ Patrones creacionales:

- ▶ Factory:

- ▶ Para poder seleccionar que implementación utilizar, se adiciona un tipo de Factoría (Factory), la cual es adicional al patrón, por lo cual no es parte del patrón pero si útil incluirla.
- ▶ Lo que nos soluciona es cual de las implementaciones debemos utilizar de una manera dinámica. Como lo realiza, instanciando la implementación y retornándola.

PATRONES DE DISEÑO

- ▶ Patrones creacionales:
- ▶ Factory – Ejemplo :

```
public class FactoriaDAO
{
    public static InterfaceDAO getDAO() {
        if (baseDatos.equals("oracle")
            return new ImplementacionDAOOracle (parametrosConexionOracle);
        else if (baseDatos.equals("MySQL")
            return new ImplemetancionDAOMySQL (parametrosConexionMySQL);
        ...
    }
}
```

PATRONES DE DISEÑO – DAO+FACTORY

► *root project*

| ... *Src*

| ... *edu.usal*

| ... *negocio*

| | ... *dao*

| | | ... *factory*

| | | ... *interfaces*

| | | ... *implementación*

| | | ... *implementacion 1*

| | | ... *implementación N*

| | ... *dominio*

| ... *util*

